

Familiar C++ Patterns That Fail at Scale and the Design Shifts That Prevent Them

DEVPRIYA DAVE &
DIVYA CHANDRASEKAR

Familiar C++ Patterns That Fail at Scale and the Design Shifts That Prevent Them

CppCon 2026
September 14, 2026

Divya Chandrasekar, Software Engineering Team Lead
Devpriya Dave, Senior Software Engineer

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Engineering

Bloomberg

Hello!



Divya Chandrasekar
Engineering Team Lead, Bloomberg



Devpriya Dave
Senior Software Engineer, Bloomberg

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

One Very Ordinary Tuesday



← Years →

↑
Ordinary
Tuesday

"Some bugs
only exist when
you zoom out"



THE NEXT HOUR

01 THE SYSTEM
What it does

02 THREE TIMES IT OUTGREW ITSELF
Three incidents

03 THREE SHIFTS
What we changed

04 WHERE THAT LEAVES US
The same patterns, in brand new world with agents

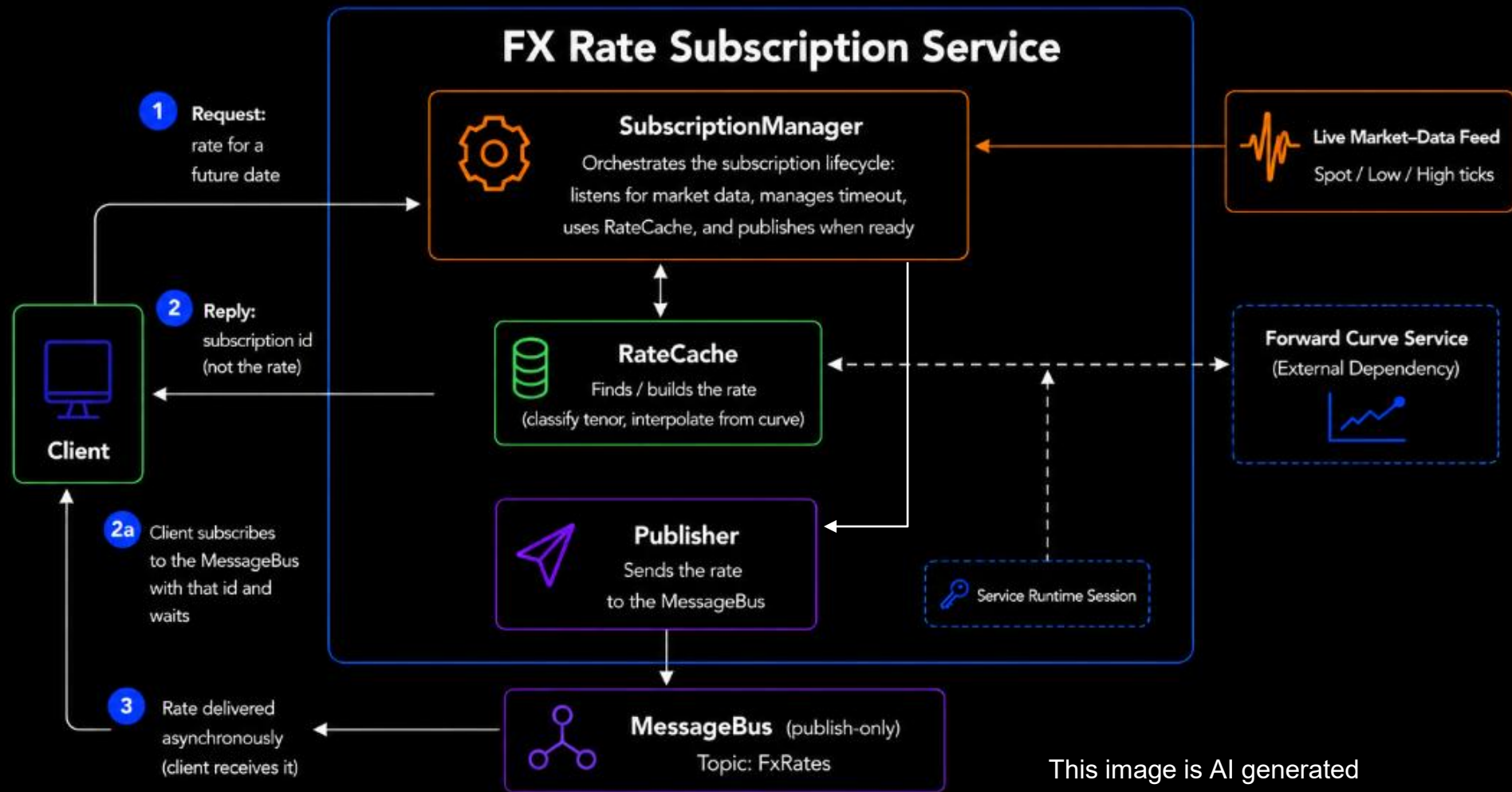
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

The Mock System Under the Microscope



What Changed as the System Scaled?

When this was written

Objects were mostly data - teardown was trivial; destruction timing didn't matter

Few components, few dependencies - global access was easy to reason about.

APIs had one or very few callers - usage conventions were easy to preserve

What changed

Objects acquired resources with explicit lifetime and teardown obligations

More components and dependencies - global access made the dependency graph harder to see, test, and change.

The same APIs were reused in broader workflows - assumptions that once lived in convention stopped holding

PATTERN

01

Shared ownership

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Everything was fine... until main() returned

Crash stack trace

```
runtime::endSession(int)
→ acquireSession()::{lambda(int)}
std::function<void(int)>::operator()()
ScopedId::{lambda(void*)}
std::_Sp_counted_deleter<...>::_M_dispose()
std::shared_ptr<void>::~~shared_ptr()
→ ScopedId::~~ScopedId()
→ RateCache::~~RateCache()
std::shared_ptr<RateCache>::~~shared_ptr()
→ __run_exit_handlers
_start
```

Why is an object calling
back into our runtime
after main has finished
using that runtime?

The reasonable-looking code

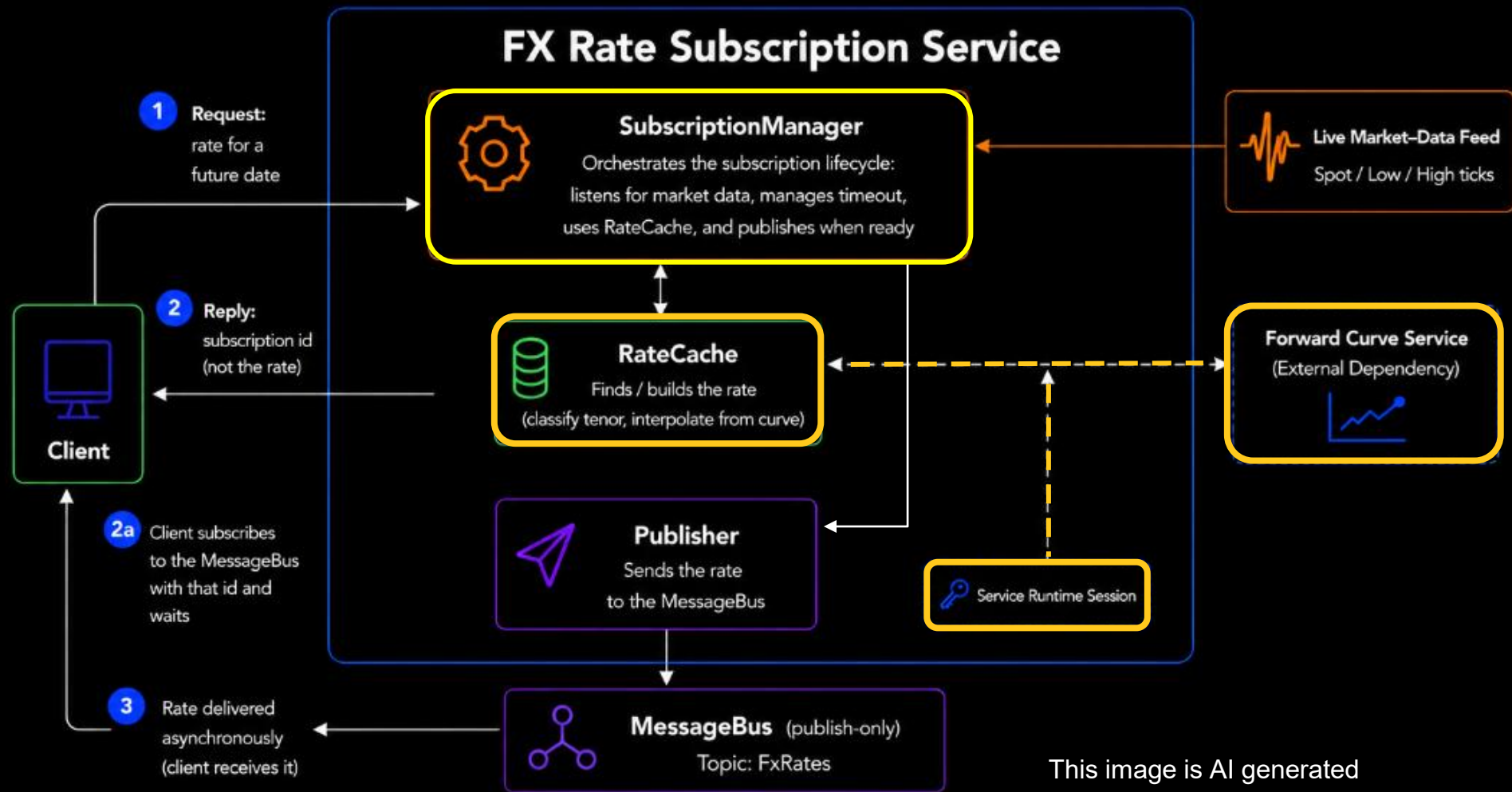
```
int main() {
    std::setvbuf(stdout, nullptr, _IONBF, 0);
    runtime::start();// starts taking in requests
    {
        App app;
        app.run();
    }
    runtime::stop();
    return 0;
}

class App {
    SubscriptionManager d_subManager;
public:
    void run() {
        runtime::serve([this](const RequestType& request)
        {
            d_subManager.clientSubscribe(request));
        });
    }
};
```

The reasonable-looking code

```
class SubscriptionManager {  
    ...  
public:  
    std::string clientSubscribe(const RequestType& request) {  
        RateCache::instance().interpolate(request);  
        ...  
    }  
};  
  
class RateCache {  
    ScopedId d_scopedId;  
    → RateCache() : d_scopedId(acquireSession()) {}  
    inline static std::shared_ptr<RateCache> d_instance;  
public:  
    ~RateCache() {}  
    → static RateCache& instance() {  
        if (!d_instance) d_instance.reset(new RateCache());  
        return *d_instance;  
    }  
    void interpolate(const RequestType&) const {}  
};
```

The Mock System Under the Microscope



This image is AI generated

Where the landmine actually hides

```
inline ScopedId acquireSession() {  
    int sessionId = runtime::openSession();  
    return ScopedId(sessionId, [](int id){ runtime::endSession(id); });  
}
```

Needs the runtime on scopeld destruction

```
class ScopedId {  
    → std::shared_ptr<void> d_guard;  
public:  
    ScopedId() = default;  
    → ScopedId(int sessionId, std::function<void(int)> onLastRef)  
        : d_guard(nullptr, [sessionId, onLastRef](void*){ onLastRef(sessionId); }) {}  
};
```

An injected callback that runs on last-ref!

So what happened?

runtime::endSession(int)

↑

acquireSession()::{lambda(int)#1}::operator()(int)

↑

ScopedId::ScopedId(...)::{lambda(void*)#1}::operator()(void*)

↑

std::shared_ptr<void>::~~shared_ptr()

↑

session::ScopedId::~~ScopedId()

↑

RateCache::~~RateCache()

↑

shared_ptr<RateCache>::~~shared_ptr()

↑

__run_exit_handlers

~ScopedId drops the last
reference to its
shared_ptr<void> guard, which
fires the deleter lambda
captured back in
acquireSession, which calls
runtime::endSession

← hidden member teardown

← destructor body is empty

← last owner released

← main() already returned

Dependency inversion: ~RateCache needs the runtime alive, but the global outlives it.

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

The Quick Fix

```
class RateCache {
    ScopedId d_scopedId;
    RateCache() : d_scopedId(acquireSession()) {...}
    inline static std::shared_ptr<RateCache> d_instance;
public:
    ~RateCache() { }
    static RateCache& instance() {...}
    static void release() { d_instance.reset(); }
    void interpolate(const RequestType&) const {}
};
```

Quick Fix

```
class App {
    SubscriptionManager d_subManager;
public:
    ~App() { RateCache::release(); }
    void run() { /* ...as shared above... */ }
};
```

The redesign: Let the type own the lifetime

```
class SubscriptionManager {  
    std::unique_ptr<RateCache> d_rateCache;  
    int d_seq = 0;  
public:  
    SubscriptionManager() : d_rateCache(std::make_unique<RateCache>()) {}  
    std::string clientSubscribe(const RequestType& request) { ... }  
};
```

- Single owner, deterministic teardown.
- ~SubscriptionManager resets the pointer
- ~RateCache runs while the runtime is alive.
- Clear lifetime guarantee, no static, no manual release() needed.

Can we do better?

```
class RateCache {  
    ScopedId d_scopedId;  
public:  
    RateCache() : d_scopedId(acquireSession()) {}  
    ~RateCache() {}  
    RateCache(const RateCache&) = delete;  
    RateCache& operator=(const RateCache&) = delete;  
    RateCache(RateCache&&) = delete;  
    void interpolate(const RequestType&) const {}  
};
```

```
[[nodiscard]] RateCache makeRateCache() { return RateCache(); }
```

```
class SubscriptionManager {  
    RateCache d_rateCache = makeRateCache();  
    ...  
};
```

Trivia Time

Why was `std::auto_ptr` deprecated in C++11 & eventually removed in C++17?

`std::auto_ptr` disguised a move as a copy.

One more: An ownership issue, not a lifetime issue

No `std::move`, no cast -- but this drains `request`;

```
void RequestProcessor::onSubscribeRequest(OwnningPtr<RateRequest>& request)
{
    try {
        std::string monitorId;
        std::shared_ptr<RateRequest> requestPtr(request);
        subscriptionManager.clientSubscribe(requestPtr, monitorId);
    }
    catch (const std::exception& e) {
        log.error("subscribe failed for {}: {}", *request, e.what());
    }
}
```

Looks Like a Copy. Acts Like a Move!

Observe & Don't Own

```
void RequestProcessor::onSubscribeRequest(OwnningPtr<RateRequest>& request)
{
    try {
        std::string monitorId;
        subscriptionManager.clientSubscribe(*request, monitorId);
    }
    catch (const std::exception& e) {
        log.error("subscribe failed for {}: {}", *request, e.what()); // still owned & safe
    }
}
```

Observe by default. Own only when the lifetime requirement says you must.

SHIFT

01

Shared ownership → **explicit lifetime**

→ Rule: Make ownership explicit.

→ Next: Let's see what pattern we discovered when we tried to add unit test

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

PATTERN

02

Global coupling via Singletons

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

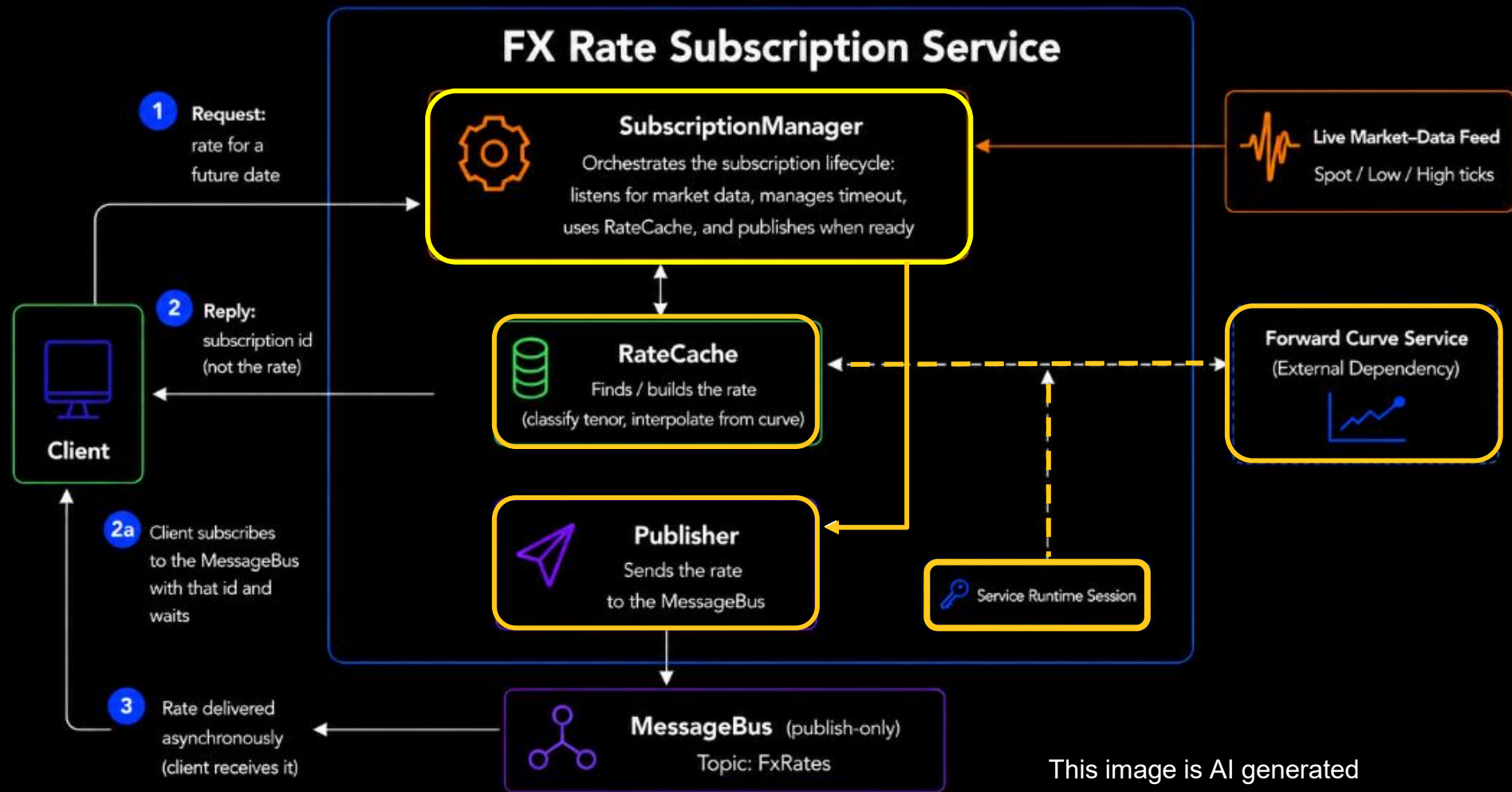
Engineering

Singleton was mainstream, respected, and widely taught, not obviously reckless code.

Gang of Four · 1994 → Meyers → Alexandrescu

Singleton was a recommendation

The Mock System Under the Microscope



This image is AI generated

The class signature doesn't reveal its dependencies.

```
class SubscriptionManager {  
    SubscriptionManager() = default;  
    inline static std::shared_ptr<SubscriptionManager> d_inst;  
public:  
    static SubscriptionManager& instance(){ ... }  
    std::string clientSubscribe(const RequestType& r){ ... }  
    void onPriceArrival(const PriceUpdate& p){  
        for (const auto& pend : d_pending) if (pend.request.pair == p.pair) {  
            double rate = RateCache::instance().interpolate(pend.request, p);  
            Publisher::instance().publishRate(pend.id, rate);  
        }  
    }  
};
```

Hidden dependency on RateCache

Hidden dependency on Publisher

RateCache singleton hides its own dependencies

```
class RateCache {
    ScopedId d_sessionId;
    RateCache(): d_sessionId(acquireSession()){}
    inline static std::shared_ptr<RateCache> d_instance;

public:
    static RateCache& instance(){...}

    double interpolate(const RequestType& r, const PriceUpdate& p) const {
        auto& cal = App::instance().calendar();
        ...
    }
};
```

SessionId depends on the ambient runtime

Reaches globals for another dependencies.

Test: Pretend this is a unit test of clientSubscribe

```
// unit test: subscribe → price arrives → publish
runtime::start();
SubscriptionManager::instance().clientSubscribe(request);
// Expected: no publication yet.
SubscriptionManager::instance().onPriceArrival({ "EURUSD", 1.0921 });
// Expected: one publication for subId.
```

- [runtime] started
[connection] backend connection opened
[calendar] calendar loaded over the connection
- [bus] LIVE publish sub-1 = 1.0942

Make the dependencies visible - inject them

```
struct IPublisher {
    virtual void publishRate(const std::string&, double) = 0;
    virtual ~IPublisher()=default; };

struct IRateCache {
    virtual double interpolate(const RequestType&, const PriceUpdate&) const = 0;
    virtual ~IRateCache()=default; };

    class SubscriptionManager {
        std::unique_ptr<IRateCache> d_rateCache;
        IPublisher&                d_publisher;
    public:
        SubscriptionManager(std::unique_ptr<IRateCache> rateCache, IPublisher& publisher)
            : d_rateCache(std::move(rateCache)), d_publisher(publisher) { ... }
        std::string clientSubscribe(const RequestType& r) { ... }
        void onPriceArrival(const PriceUpdate& p) { ... }
    };
```

Make the dependencies visible - inject them

```
struct ISettlementCalendar {  
    virtual int tradeDate(const RequestType&) const = 0;  
    virtual ~ISettlementCalendar()=default;  
};  
  
class RateCache : public IRateCache {  
    ScopedId d_sessionId;  
    ISettlementCalendar& d_calendar;  
public:  
    explicit RateCache(ISettlementCalendar& cal):d_sessionId(acquireSession()),d_calendar(cal){...}  
    double interpolate(const RequestType& r, const PriceUpdate& p) const override {...}  
};
```

Now that the seam exists, you can test it

```
struct FakeRateCache : IRateCache {  
    double interpolate(const RequestType&) const override { return 1.5; }  
};  
struct FakePublisher : IPublisher {  
    std::string lastId;  
    double lastRate = 0;  
    void publishRate(const std::string& id, double r) override {  
        lastId = id;  
        lastRate = r;  
    }  
};
```

Canned rate, no runtime
needed

capture instead of publish

```
/* Test */  
FakePublisher pub;  
SubscriptionManager mgr{ std::make_unique<FakeRateCache>(), pub };  
auto subId = mgr.clientSubscribe(request);  
assert(pub.lastId == subId && pub.lastRate == 1.5);  
[test] captured sub-1 = 1.5000 -> PASS (no runtime, no backend)
```

Inject both the fakes

Wire it once: The composition root

```
int main(){
    std::setvbuf(stdout, nullptr, _IONBF, 0);
    runtime::start();
    {
        Connection          connection;
        SettlementCalendar calendar{connection};
        Publisher            publisher;
        SubscriptionManager mgr{ std::make_unique<RateCache>(calendar), publisher };
        App app{mgr};
        app.run();
    } // reverse-order teardown, runtime still up
    runtime::stop();
    return unit_test();
}
```

SHIFT

02

Singletons → Injected dependencies

- Rule: Make dependencies visible
- Next: Let's see what pattern we discovered when we tried add more instances

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

PATTERN

03

Permissive APIs

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

"Some bugs
only exist when
you zoom out"



Let's look at this “upsert”



```
void SubscriptionRepository::save(const RateSubscription& sub)
{
    if (updateExistingRow(sub)) {
        return;
    }

    if (!insertNewRow(sub)) {
        throw std::runtime_error("DB Error.");
    }
}
```

update → otherwise insert → otherwise fail

Two correct requests collided

```
Instance 1: UPDATE subscription_id = X -> 0 rows  
Instance 2: UPDATE subscription_id = X -> 0 rows  
Instance 1: INSERT subscription_id = X -> success  
Instance 2: INSERT subscription_id = X -> duplicate key  
Instance 2: throw "DB Error." -> client request fails
```

The second request wasn't wrong. The stored row wasn't wrong.

Yes, fix the race in the database

“Use a unique constraint. Use INSERT ... ON CONFLICT.”

Fine. The database is now happy.

But the caller still can't tell "I created this"
from "it was already there."

A Fix: Verify Before Failing

```
void SubscriptionRepository::save(const RateSubscription& sub)
{
    if (updateExistingRow(sub)) {
        return;
    }

    if (!insertNewRow(sub)) {
        if (sameSubscriptionExists(sub)) {
            return;
        }
        throw std::runtime_error("DB Error.");
    }
}
```

We discovered a meaningful state, handled it, and then erased it again before returning to the caller.

What can this function actually tell us?

```
void save(const RateSubscription& sub);
```



The system has these states, but the API signature does not convey that

Is 'already done' an error?

It depends on the caller.

The API never gives the caller the chance to know.

Make the outcome part of the API

```
enum class SaveResult {  
    Updated,  
    Inserted,  
    AlreadyPresent  
};
```

```
struct SaveError {  
    SubscriptionId id;  
    std::string reason;  
};
```

```
[[nodiscard]]
```

```
std::expected<SaveResult, SaveError> save(const RateSubscription& sub)
```

What if we don't own an API?



and a Legal Choice Breaks Your Assumptions

[Message Bus]: FATAL Assertion failed: false && "CorrelationId must be unique"

Client Service: Creating and Tracking a Rate Subscription

```
class Consumer {
    std::map<CorrelationId, std::shared_ptr<Topic>> d_subs; ...
    CorrelationId watch(const std::string &rateKey) {
        auto topic = std::make_shared<Topic>(Topic{rateKey, {}});
        CorrelationId corrId(topic.get());
        d_bus.subscribe(corrId, *topic);
        d_subs.insert({corrId, topic});
        return corrId;
    }
    void onRate(CorrelationId corrId, double rate) {
        const Topic *topic = static_cast<const Topic *>(corrId.getAsPointer());
        const std::string key = topic->rateKey;
        // handle rate ...
        d_bus.unsubscribe(corrId);
        d_subs.erase(corrId);
    }
};
```

What happened: A Classic Memory Reuse Failure

```
// a client needs EURUSD for 15 Sep
[consumer] watch EURUSD/20260915 Topic at 0x...510 -> id = 0x...510
// the rate arrives; the consumer is done with that subscription
[consumer] rate 1.0921 for "EURUSD/20260915"
[consumer] done -- asked bus to drop 0x...510, Topic freed
// the broker has not processed the unsubscribe yet
// next client needs USDJPY for 20 Nov; 0x...510 is reassigned
[bus] !! Assertion failed: CorrelationId must be unique (id = 0x...510)
Aborted
```

```
// A Permissive API with 4 legal choices
CorrelationId(long long n);
CorrelationId(const void *p);
CorrelationId(std::shared_ptr<void> p);
CorrelationId(AutoValue a);
```

The Fix: Stop Using Object Address as Identity

```
class Consumer {  
    CorrelationId watch(const std::string &rateKey) {  
        Topic topic{rateKey, {}};  
        CorrelationId corrId(CorrelationId::AutoValue::create());  
        d_bus.subscribe(corrId, topic);  
        d_subs.insert({corrId, rateKey});  
        return corrId;  
    }  
    void onRate(CorrelationId corrId, double rate) {  
        auto it = d_subs.find(corrId);  
        if (it == d_subs.end())  
            return;  
        // handle rate  
        d_bus.unsubscribe(corrId);  
    }  
};
```

// Erase only on unsubscribe confirmed my message bus

Constrain the Boundary You Own

```
SubscriptionId makeSubscriptionId() {  
    return SubscriptionId{  
        CorrelationId(AutoValue::create())  
    };  
}  
SubscriptionId Consumer::watch(const std::string& rateKey) {  
    Topic topic{rateKey, {}};  
    auto subId= makeSubscriptionId();  
    d_bus.subscribe(subId.value(), topic);  
    d_subs.insert({corrId, rateKey});  
    return subId;  
}
```

Constrain what you can...

THEIR GENERAL-PURPOSE API:

Many legal constructors

Must serve every caller

Flexibility is the feature

OUR PROTOCOL BOUNDARY:

One way to make an identity

Serves one system

Constraint is the feature

You can't narrow their API. You can narrow the door your code walks through.

Shift

03

Permissive APIs → constrained interfaces

→ Rule: Make invalid states unrepresentable.

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Does modern C++ save you?

Shift	The design failure	Modern C++ helps with	But C++ still can't decide...
Lifetime	Ownership is implicit	unique_ptr + move (C++11), [[nodiscard]] and copy elision (C++17)	Who should own it and when must it die?
Dependencies	Collaborators hidden behind globals	Factories, move-only values, Concepts (C++20)	Which dependencies are explicit and where is the graph composed?
Interfaces	API allows states the protocol can't support	enum class (C++11), optional / variant (C++17), expected (C++23), strong types	Which states should the interface allow at all?

Every one of these is vocabulary. The decision is still yours, in any language.

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Could an agent have caught these?

- Yes, it can flag a lot of these.
- Agents can propose and even implement fixes.
- But they still need to infer the intended contract (from existing code):
 - who should own this,
 - what must stay alive,
 - which dependencies belong in the interface, and which states are actually valid.
- The more explicitly we encode those decisions in the design, the less humans and agents have to guess.

YOUR TAKEAWAYS

✓ Make ownership explicit.

If you only read it, don't own it.

If ownership moves, make the transfer visible.

Lifetime/Ownership Issues: Walk the destructor chain of every static, singleton and long-lived shared ptr transitively; report each chain ending in I/O, deregistration or a framework call, quoting the chain.

✓ Make dependencies visible.

A class shouldn't discover its collaborators. Give them to it.

If a unit test needs the real system, a dependency is probably hiding.

Find Invisible dependencies: For each class, list what it uses that isn't a parameter, and report what you'd have to stand up to construct it in a unit test.

✓ Make invalid states unrepresentable.

If callers must remember a rule, encode it in the interface.

If failure is expected, make it part of the result.

Find permissive APIs: For each signature, name what it permits that is never valid; for each type with several constructors, name the obligation each imposes and which one the codebase uses.

Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Engineering

Bloomberg